RAG Assignment: Baseline vs. Re-Ranking

Maram Charan Teja, Roll No. 2025201010, 19 October 2026. Code: github.com/CharanTeja2232/RAG (notebook *RAG_Baseline_vs_Rerank.ipynb*)

Setup

I used NIST SP 800-30 Rev. 1, *Guide for Conducting Risk Assessments*, which is in the public domain. Its body text is split into 305 chunks of at most 240 tokens. Chunks never cross a heading, and each glossary entry is its own chunk. I froze the corpus by hash before writing any question.

Pipeline 1 embeds the chunks with *all-MiniLM-L6-v2* (normalised, 384 dimensions) into an exact FAISS inner-product index, retrieves the Top 10 and gives the first 3 chunks to the LLM, which produces Answer 1. Pipeline 2 scores the same 10 chunks with *cross-encoder/ms-marco-MiniLM-L-6-v2*, re-sorts them and gives the new Top 3 to the LLM with the identical prompt, which produces Answer 2. The LLM is *qwen/qwen3.8-27b* on Groq, with temperature 0 and at most 300 output tokens. The prompt tells it to answer only from the passages, to say that the context does not contain the answer when it does not, and to cite chunk ids.

There are 24 questions in four groups of 6: written as keyword traps, exceptions, negations or constraints, and buried details. I fixed the gold chunks before running any retrieval. The primary metric counts a query as a success when a gold chunk is in the Top 3. A stricter secondary rule counts only chunks that answer the question on their own, or complete joint evidence.

Result over 24 queries	Baseline	Re-ranked
Gold chunk in Top 3 (primary)	21 / 24	24 / 24
Strict rule satisfied in Top 3 (secondary)	20 / 24	24 / 24
Gold chunk at rank 1	13 / 24	19 / 24
Queries whose Top-3 set changed, demotions, evidence missing from the Top 10	22 of 24, 0, 0	

1. Architecture comparison: Bi-Encoder vs. Cross-Encoder

A Bi-Encoder produces independent embeddings: it encodes the query and each chunk separately into fixed-size vectors. Relevance is the cosine or inner product of two vectors, so the query and the chunk never see each other. A Cross-Encoder concatenates the query and one chunk into a single input and runs self-attention jointly over all their tokens. Every query word can therefore interact with every chunk word in every layer, and the model outputs one relevance score for the pair.

Aspect	Bi-Encoder (stage 1)	Cross-Encoder (stage 2)
Input and interaction	Independent embeddings; the two texts meet only in the final dot product	(query, chunk) encoded together, with attention between every pair of tokens
Output	One vector per text; the score is the cosine similarity	One relevance logit per pair (not comparable with cosines)
Precomputation	Chunk vectors computed once and indexed	None: a forward pass for every (query, chunk) pair at query time
Strength and weakness	Very fast recall, but one vector blurs negation, "except when" clauses and lexical look-alikes	Judges a pair precisely, including qualifiers and negation, but is too expensive to run on everything

2. Qualitative and rank-shift analysis

The baseline missed the Top 3 for 3 of 24 queries (A4, E3 and N5), and for E4 as well under the strict rule. In each case the chunk that answers the question sat between ranks 4 and 10, and the Cross-Encoder moved it into the Top 3. None of the six buried-detail queries failed at baseline, so the examples come from the other three categories. No query got worse after re-ranking (0 demotions).

Query	Question	Answer-bearing chunk: rank shift	Baseline Top 3	Re-ranked Top 3
A4 glossary distractor (written as a keyword trap)	Can something about an organization's environment, such as a system with no outside network connection, make a successful attack less likely?	**nt-040** #6 to #1	glossary entry "Threat", nt-038 (Vulnerabilities and predisposing conditions), glossary entry "Vulnerability"	**nt-040**, nt-041, nt-043

Query	Question	Answer-bearing chunk: rank shift	Baseline Top 3	Re-ranked Top 3
E3 exception	Can a threat event be dropped from the analysis when no adversary is known to have the ability to carry it out?	**nt-286** #7 to #2	glossary entry "Threat Event", nt-034 (Threats), nt-043 (Likelihood)	nt-034, **nt-286**, nt-039
N5 negation / constraint	Does this guidance cover systems that are designated as national security systems?	**nt-010** #4 to #1	glossary entry "National Security System", glossary entry "Security Policy", glossary entry "Tailoring"	**nt-010**, nt-227, nt-273
E4 exception (strict rule only)	When conditions have changed since the last assessment, must every task be redone?	**nt-164** #8 to #2 (nt-163 stays #1)	nt-163, nt-160, glossary entry "Repeatability" (both nt-163 and nt-164 are needed)	nt-163, **nt-164**, nt-160

In all three failures, short glossary entries (*Threat*, *Vulnerability*, *Threat Event*, *National Security System*, *Security Policy* and *Tailoring*) held places in the baseline Top 3 and pushed the answer-bearing chunk down. They are definitions of terms named in the question, so they are plausible distractors, but they are not keyword traps. I counted the question's content words that appear in each chunk, and the answer-bearing chunk shares more of them than any baseline chunk does (A4: 5 of 12 question words in the answer chunk against 0 to 1 in the baseline chunks; E3: 3 of 8 question words in the answer chunk against 2 in the baseline chunks; N5: 4 of 6 question words in the answer chunk against 1 to 2 in the baseline chunks). Across all 24 queries, no non-answer chunk in the baseline Top 3 shares more of the question's words than the best answer chunk does, so by this simple measure I found no true lexical keyword trap. I did not test why the Bi-Encoder ranks short definitions so high.

Against the failure types in the brief: E4 is the clearest exception case, because the baseline already had the general rule (nt-163) but missed the "otherwise" condition in nt-164, which sat at rank 8. N5 fits negation and constraints, because the answer is an exclusion ("other than ... national security systems") and the baseline returned definitions of the term instead. All four cases also fit the last type, a vital chunk buried at ranks 4 to 10 and promoted into the Top 3.

Answer 1 (baseline context) vs. Answer 2 (re-ranked context)

Query	Answer 1	Answer 2
A4	The provided context does not contain the answer.	Yes, a predisposing condition such as a stand-alone information system with no external network connectivity can decrease the likelihood of exposure to a network-based cyber attack [nt-040].
E3	The provided context does not contain the answer.	Yes, organizations can eliminate certain threat events from further consideration if no adversary with the necessary capability has been identified. [nt-286]
N5	The provided context does not contain the answer.	No, the guidelines in this publication are applicable to all federal information systems other than those designated as national security systems. However, they may be used for such systems with the approval of appropriate federal officials exercising policy authority over them [nt-010].
E4	No, organizations can revisit the purpose, scope, assumptions, and constraints to determine whether all tasks in the risk assessment process need to be repeated. [nt-163]	No, organizations can revisit the purpose, scope, assumptions, and constraints to determine whether all tasks need to be repeated, and otherwise, the updates constitute subsequent risk assessments that identify and assess only how selected risk factors have changed [nt-163] [nt-164].

In A4, E3 and N5 the baseline model did not hallucinate. With the key chunk outside the Top 3 it said the context does not contain the answer, which is what its prompt tells it to do when the passages lack one. The user still gets no answer, although the document contains one.

In E4, Answer 1 states only the first half of the rule: revisit the scope to decide whether all tasks must be repeated. The second half says that otherwise only the changed risk factors are re-assessed. It is in nt-164, which was at rank 8. Answer 2 states both halves. I found no misstated rule in these four pairs.

I compared answer quality only in these four cases, by reading them. The other 20 pairs were not graded for correctness. The citation counts are a format check, not a correctness check: Answer 1 cited a gold chunk in 21 of 24 answers and Answer 2 in 24 of 24, and the two answers are identical word for word in 8 of 24 pairs. Answer 2 also received more text (14,454 vs 12,784 input tokens in total, 13% more), so these results do not show that re-ranking improves answer correctness in general.

3. Latency profiling

Step (local CPU, ms; n = 480 per row)	mean	median	st. dev.	p95
(1) Query embedding (Bi-Encoder)	2.87	2.83	0.28	3.10
(2) FAISS exact Top-10 search	0.02	0.02	0.00	0.03
(3) Cross-Encoder scoring of the 10 pairs	92.60	93.03	12.34	108.20
Pipeline 1 (baseline retrieval) = (1) + (2)	**2.89**	**2.85**	**0.28**	**3.13**
Pipeline 2 (re-ranking) = (1) + (2) + (3) + sort	**95.51**	**95.95**	**12.40**	**111.10**
Re-ranking overhead (Pipeline 2 minus Pipeline 1)	92.62	93.04	12.34	108.21

Measured on an Apple M5 with 16 GB, on the CPU with a single thread (a macOS OpenMP clash between torch and FAISS forced *OMP_NUM_THREADS=1*). The models were already loaded. I discarded 3 warm-up passes, then timed 20 passes over the 24 queries, with other processes running. One-time costs are excluded: loading the Bi-Encoder took 0.029 s, loading the Cross-Encoder 0.022 s from the local cache, and embedding all 305 chunks 1.711 s. Every timed pass reproduced the stored Top 10 and re-ranked order exactly.

Re-ranking adds about 93 ms per query, so Pipeline 2 takes roughly 33 times as long as Pipeline 1, and the Cross-Encoder step accounts for almost all of it. The cost varies with passage length: per-query medians of Pipeline 2 range from 51 to 111 ms.

I timed LLM generation separately. Each call went to a remote API, so network and queueing are included. Answer 1 had a median of 326 ms (mean 423) and Answer 2 a median of 316 ms (mean 397), over 24 calls each. These numbers come from different hardware, a network and far fewer samples than the local timings. They cannot be compared with or added to the local timings, and I claim no latency difference between the two answers.

4. Scaling question: why not run the Cross-Encoder over the whole collection?

A Cross-Encoder needs the query and a chunk together in one forward pass, so nothing about a chunk can be precomputed or indexed. The cost per query grows linearly with the number of chunks, and every query pays it in full. A Bi-Encoder embeds the chunks once, offline, and each query then costs one short encoding plus a vector search (exact here, sub-linear with approximate indexes). I measured the difference: scoring all 305 chunks took a median of 1,952 ms per query (6.4 ms per pair), 21 times the cost of scoring only the Top 10, and this corpus has only 305 chunks. A linear projection, which I did not measure, gives about 64 s per query at 10,000 chunks and about 6,399 s (about 1.8 hours) at 1,000,000 chunks. That is why the design has two stages: a cheap Bi-Encoder pre-filter with high recall narrows the collection to K = 10 candidates, and the expensive Cross-Encoder runs only on those.

The re-ranker can only reorder its candidates, so a chunk that the Bi-Encoder leaves outside the Top 10 is lost for good. That did not happen here (0 of 24 queries), and it is the reason K must be large enough.

Conclusion

On this document the re-ranker moved all three missed chunks into the Top 3 (A4 from rank 6 to 1, E3 from rank 7 to 2, N5 from rank 4 to 1), demoted no query, and added about 93 ms per query on one CPU thread. When the key chunk was missing, the baseline gave no answer (or, for E4, left out a clause) and did not hallucinate. Running the Cross-Encoder over the whole collection would not scale. This small study does not show how much re-ranking improves answer quality beyond these cases.

Limitations

- The evaluation is small. N = 24 with only 3 baseline failures (4 under the strict rule), so I make no significance claim. The questions, gold labels and scoring rules were written before retrieval by one author with an AI assistant, and nobody else validated them.
- The questions written as keyword traps reuse the source's own wording, so they did not work as traps, and the buried-detail questions produced no baseline failure.
- Each query and pipeline has one generation, from a preview model. The API served the 48 calls from 4 different backend builds, so small wording differences between Answer 1 and Answer 2 should not be over-read.
- The repository holds the hashes, the settings, the 48 answers with their metadata and 37 automated integrity checks.